

BLE (3) - ATT/GATT 이해하기

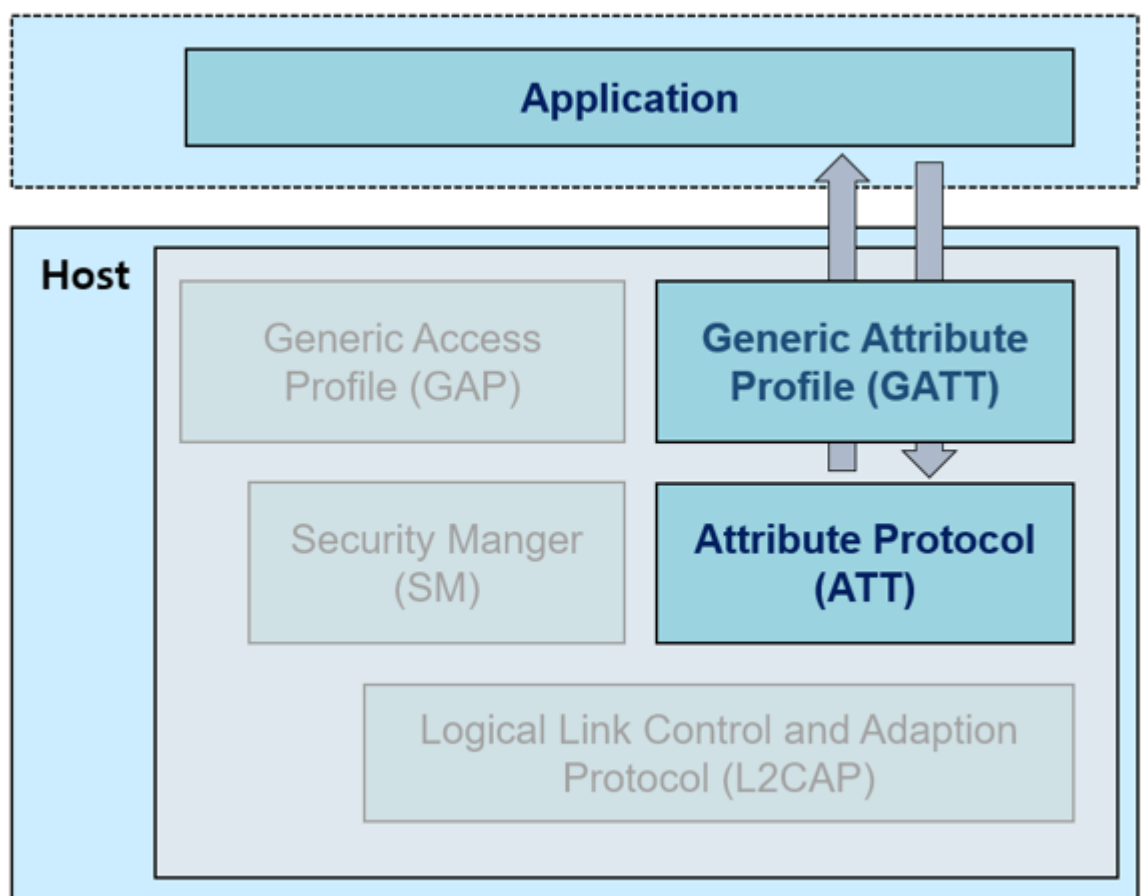
Updated: April 16, 2020

On this page

- 3. ATT/GATT
 - 3.1 GATT
 - 3.2 Attribute Table 예시
 - 3.3 Service 정의 순서
 - 3.4 GATT Server/Client

3. ATT/GATT Permalink

BLE 프로토콜 스택에서 Attribute protocol (ATT)은 서버 (server)와 클라이언트 (client) 사이의 데이터 교환에 대한 규칙을 정의한다. 다음 그림에 묘사된 것 같이, 어플리케이션 단에서의 데이터 교환은 ATT를 기반으로 이뤄지며 각각의 데이터는 구조는 Generic Attribute Profile (GATT)에 의해 정의되는 데이터 구조를 따른다.

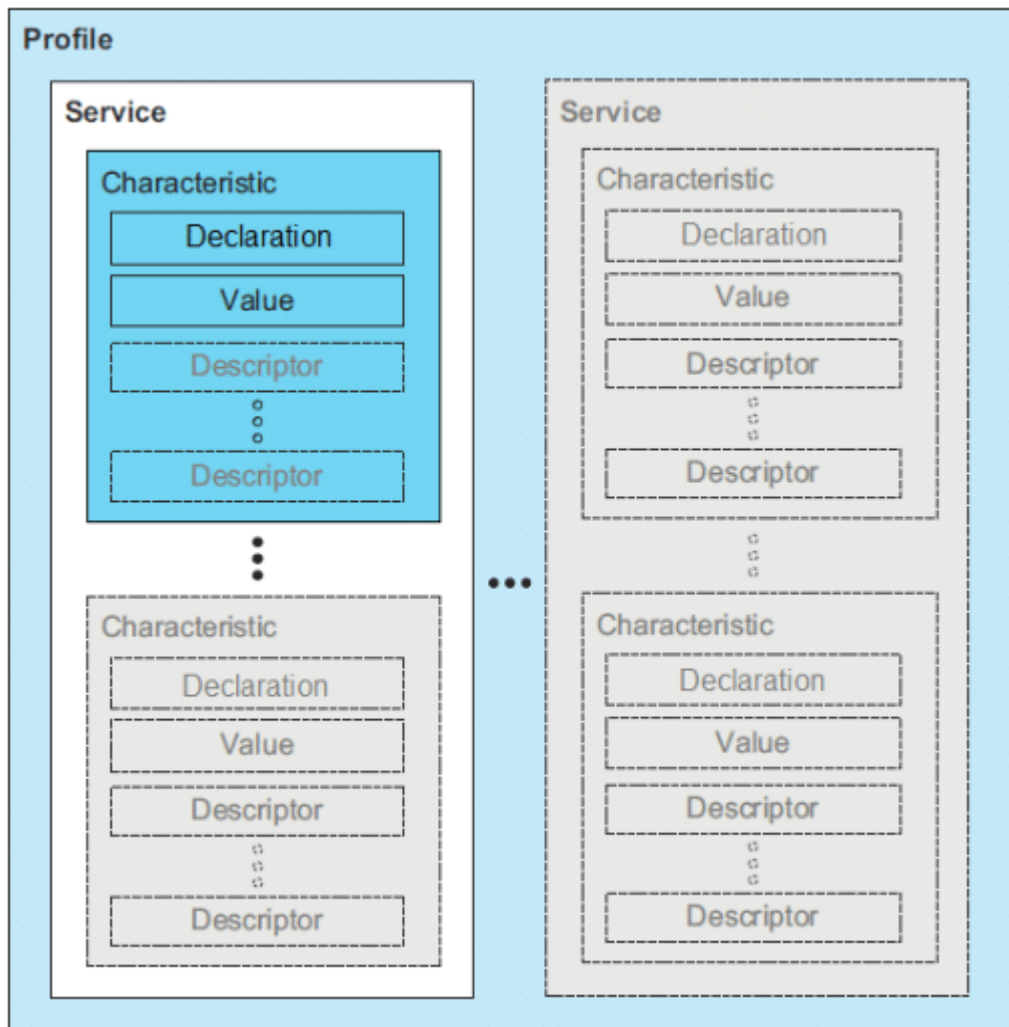


대부분의 BLE 칩 제작사 (e.g. Nordic Semiconductor, Texas Instruments, .etc)는 BLE 프로토콜 스택 소스를 제공하기 때문에, 펌웨어 개발자가 직접 BLE 스택을 프로그래밍하거나 수정하는 경우는 드물다. **다만, BLE 디바이스의 데이터가 어떤 식으로 생성되고, 어떤 정보를 기반으로 데이터를 교환하는지 알아두는 것은 전반적으로 BLE 프로토콜을 이해하는데 큰 도움이 될 것이다.**

3.1 GATTPermalink

GATT 에 의해 정의되는 BLE 시스템의 데이터 구조는 다음의 그림과 같이 **service** 와 **characteristic** 으로 표현된다.

Service is a collection of information, and **Characteristic** is a set of actual values and information



출처:

https://www.researchgate.net/figure/BLE-Data-Hierarchy_fig2_317553032

Service 는 특정 어플리케이션을 구현하기 위해 필요한 데이터의 집합이라고 할 수 있으며, 각각의 데이터는 **characteristic** 에 의해 정의된다. 예를 들어, 심박수 측정 데이터를 관리하기 위한 **service** 를 정의할 경우, 해당 **service** 는 실제 심박 수치를 저장하는 **characteristic** 을 비롯해, 센서의

단위나 모델 번호, 또는 보안 관련 파라미터 등의 정보를 관리하는 다수의 **characteristic** 를 포함할 수 있다.

이러한 **service** 와 **characteristic** 에 대한 정보는 **attribute** 라는 최소 데이터 유닛에 의해 정의되고, BLE 디바이스 내의 **attribute** 에 대한 정보는 최종적으로 **attribute table** 내에 저장된다.

각각의 **attribute** 는 다음의 세 가지 정보를 포함하고 있다.

- type (UUID)
- handle
- permission

3.2 Attribute Table 예시 [Permalink](#)

다음은 **attribute table** 의 예시를 보여주며, 각 행(row)은 하나의 **attribute** 를 나타낸다.

Attribute Table				
Type	Handle	UUID	Permission	Value
Service declaration	0x000C	0x2800 (Service declaration)	Read Only, No Authentication No Authorization	0x180D (HRS)
Char. dec.	0x000D	0x2803 (Char. dec.)	Read Only, No Authentication No Authorization	Char. Info. (... UUID: 2A37, value handle: 0x000E, ...)
Char. val.	0x000E	0x2A37 (Spec. char. value)		167 Beats per min.
Char. dec.	0x000F	0x2803		

handle 은 **attribute** 의 주소 값에 해당되고, permission 에는 단어 뜻 그대로 해당 **service** 또는 **characteristic** 의 접근 권한에 대한 정보 등이 저장된다. UUID 는 **Universally Unique ID** 의 약자로, **GATT** 데이터 성분을 구분하기 위한 고유 식별자로 사용된다.

3.3 Service 정의 순서 [Permalink](#)

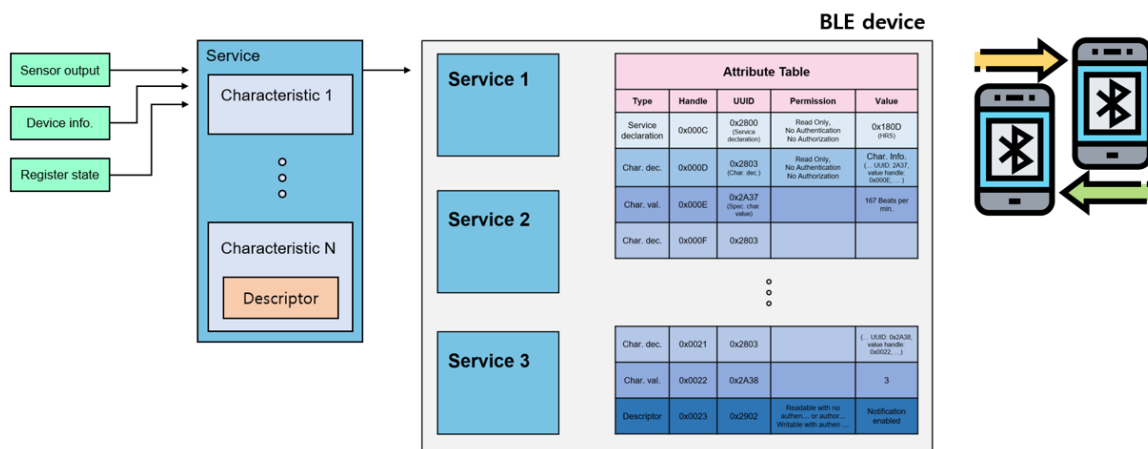
0x2800 은 **service** 선언에 대한 **attribute UUID** 값이다. 위의 **attribute table** 그림에서 맨 위쪽 행을 보면, **UUID = 0x2800** 값을 갖는 **attribute** 를 이용해 **service** 를 선언하고 있다. 본 예시에서 해당 **attribute** 의 **value** 에는 심박(Heart Rate) 측정 **service** 에 대한 **UUID** 값 0x180D 이 저장되어 있음을 볼 수 있다.

Bluetooth SIG에서는 자주 사용되는 **service** 에 대한 **UUID** 를 사전에 정의해두고 있으며, 이러한 **service** 의 경우 **16 bits** 사이즈의 **UUID** 값을 갖는다. 사전에 정의된 **GATT service** 에 대한 **UUID** 값은 [본 링크](#)에서 확인 가능하다.

다음으로, 0x2803 은 **characteristic** 선언에 대한 **attribute UUID** 값이고, 해당 **attribute** 의 **value** 에는 선언하고자 하는 **characteristic** 의 **UUID = 0x2A37** 와 **handle** 값이 저장되어 있다. 0x2A37 은 **HRS** 에서 심박 수 센서 값을 저장하는 **characteristic** 의 **UUID** 이고, 위의 예시에서 볼 수 있듯이 **UUID = 0x2A37** 값을 가지는 **attribute** 의 **value** 에는 167 이란 정보가 저장되어 있는 것을 볼 수 있다.

BLE 디바이스는 하나 이상의 **service** 를 포함할 수 있으며, 각각의 **service** 또한 하나 이상의 **characteristic** 을 포함할 수 있다.

다음은 **BLE 디바이스** 내부의 데이터 구조의 예시를 보여준다.



3.4 GATT Server/Client [Permalink](#)

무선 연결된 **BLE 디바이스** 중 데이터를 가지고 있는 디바이스가 **GATT server** 가 되고, 데이터를 요청하는 디바이스가 **GATT client** 가 된다.

예를 들어, 스마트폰 APP 을 이용해서 **BLE** 센서 노드 (e.g. 센서 데이터 수집 후, **BLE** 프로세서를 기반으로 데이터를 송신하는 디바이스)의 데이터를 수신하고자 할 때, **BLE** 통신을 시작하는 디바이스는 스마트폰이기 때문에 스마트폰이 **GAP central** 이 되고, 센서 노드는 **GAP peripheral** 이 된다. 그리고 실질적인 데이터를 가지고 있는 **BLE** 센서 노드는 **GATT server** 가 되고, 데이터를 요청하는 스마트폰은 **GATT client** 가 된다.

Reference

https://www.researchgate.net/figure/BLE-Data-Hierarchy_fig2_317553032

<https://devzone.nordicsemi.com/nordic/short-range-guides/b/bluetooth-low-energy/posts/ble-characteristics-a-beginners-tutorial>